ConvPE: Convolutional Position Embedding Improves Transformers

Wang Zehao
Nanjing University
wzhagi@163.com

ABSTRACT

ConvPE (Convolutional Position Embedding) substantially improves the performance and efficiency of Transformer models. With only 2.64% more parameters and a 2.71% increase in forward-pass time, it reduces training perplexity by 8.41%. Crucially, it achieves the baseline's final loss with 31.33% fewer training tokens, yielding a 26.87% net reduction in wall-clock training time. ConvPE also exhibits superior context extrapolation, increasing RULER accuracy by 29.51% at 1K and 314.6% at 2K contexts. By generating windowed attention-score biases via a lightweight convolution, ConvPE remains compatible with KV-cache concatenation, various attention masks, and high-dimensional modalities. The code is publicly available at `https://github.com/wzhagi/convpe`.

Keywords Position Embedding · Language Models · Transformers · Attention · Vision Transformers

1 Introduction

Transformers [1] are insensitive to input order, whereas word order is crucial for language understanding. Explicit or implicit position embeddings are therefore necessary to inject order information into the model [2,3,4].

This paper proposes ConvPE, a simple and efficient position-embedding scheme that generates windowed attention-score biases with a lightweight convolution. This design allows ConvPE to switch freely between different attention masks, be combined with RoPE, support KV-cache concatenation, and naturally extend to high-dimensional modalities with n-dimensional convolution.

We comprehensively evaluate ConvPE in language-model pretraining and vision transformers. As summarized in Figure 1, ConvPE improves performance and efficiency across the board.

Performance:

ConvPE reduces training perplexity by 8.41% and improves average evaluation accuracy by 2.35%. It also transfers effectively to vision tasks, improving ViT-Tiny [5] accuracy on CIFAR-10 and CIFAR-100 [6] by 2.31% and 10.85%.

Training and Token Efficiency:

Despite a small 6.54% per-token wall-clock overhead and 2.64% more parameters, ConvPE reaches the RoPE baseline's performance with 31.33% fewer training tokens. This results in a net 26.87% reduction in total wall-clock training time to match the baseline.

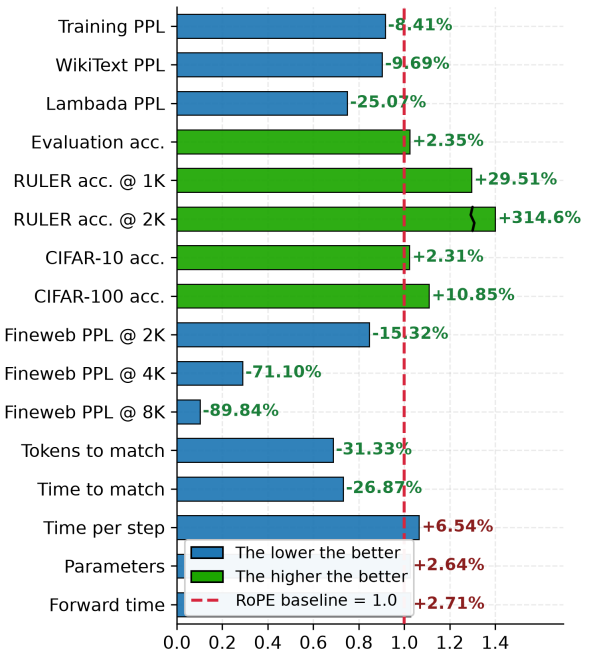


Figure 1: ConvPE / RoPE ratios.

Context Extension:

ConvPE significantly mitigates performance degradation at zero-shot context extensions. Compared to RoPE, it reduces perplexity on 1M tokens from the training dataset by 15.32% at 2K, 71.10% at 4K, and 89.84% at 8K. On the RULER benchmark, ConvPE boosts accuracy by 29.51% at 1K and by over 300% at 2K contexts.

The main contributions of this paper are as follows:

- We propose ConvPE, a convolutional attention-score-bias position embedding that combines strong performance with broad compatibility.
- We study the influence of ConvPE hyperparameters and architecture choices, identifying a simple default configuration with a highly favorable performance-efficiency balance.
- We demonstrate that ConvPE significantly improves training efficiency, token efficiency, and zero-shot long-context extrapolation in language models.
- We verify the effectiveness of ConvPE2D in vision transformers on CIFAR-10 and CIFAR-100.
- We develop high-performance TileLang[7] kernels for ConvPE.

2 Background

2.1 Attention Mechanism

Given the input query $Q \in \mathbb{R}^{S_q \times D_k}$, key $K \in \mathbb{R}^{S_{kv} \times D_k}$, and value $V \in \mathbb{R}^{S_{kv} \times D_v}$, the computation of scaled dot-product attention[1] $O \in \mathbb{R}^{S_q \times D_v}$ is:

$$O = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \mathbf{B}\right) V$$

S_q is the query sequence length. S_{kv} is the key/value sequence length. $\mathbf{B} \in \mathbb{R}^{S_q \times S_{kv}}$ is the attention score bias. Causal models introduce a causal mask $M \in \mathbb{R}^{S_q \times S_{kv}}$ that sets the attention score bias $\mathbf{B}$ for invisible positions to $-\infty$. With batch size and multi-head settings, Q, K, V, O additionally have B, H dimensions.

2.2 Position Embedding

Existing position embedding schemes can be roughly divided into five categories:

1. **Absolute Position Embedding**[1]: adds a fixed positional bias, usually sine waves, to the input vector. It is simple to implement, but its performance is usually limited.
2. **Learnable Positional Bias**[2,3]: adds a learnable positional bias to the input sequence. It is typically adopted in limited-window settings, such as spatial dimensions in vision models or fixed-rate compression.
3. **Rotary Position Embedding**[4]: encodes position as a rotation matrix and applies it to queries and keys, so that attention scores depend on relative positions. RoPE has been widely adopted in modern large language models, enabling long-context extension via frequency scaling, such as YaRN[8].
4. **Attention Score Bias**: adds a positional bias B to attention scores[9,10,11]. RoPE can be viewed as a rotation-based equivalent, as it also makes attention scores depend on relative positions.
5. **No Position Embedding (NoPE)**[12]: adds no explicit position embeddings; instead, it relies on model structure and causal attention masks to implicitly learn positional information requiring no explicit Position Embedding[13].

3 Method

Given token representations $X \in \mathbb{R}^{B \times S \times D}$, ConvPE first projects them to a small channel dimension $X_{down} \in \mathbb{R}^{B \times S \times C}$, then uses a convolution to generate windowed attention score bias $\mathbf{B}_{ConvPE} \in \mathbb{R}^{B \times S \times H_b \times W}$ for self-attention keys within the convolution window as shown in Figure 2.

$$\mathbf{B}^{b,h,s_q,s_{kv}} = \begin{cases} \mathbf{B}_{\text{ConvPE}}^{b,s_q,h \bmod H_b,w}, & w \in [0, W-1] \text{ where } w = s_{kv} - s_q + W - 1 \\ 0, & \text{otherwise.} \end{cases}$$

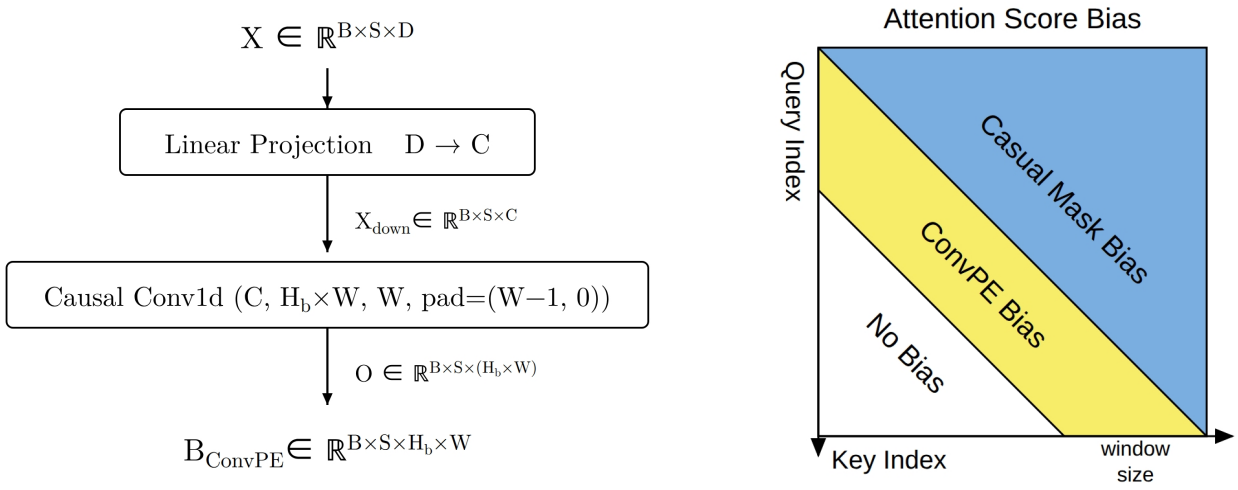


Figure 2: ConvPE Algorithm

The hyperparameters are:

- W: convolution window size.
- C: out features of linear projection and in channels of convolution.
- H_b: number of score bias heads.

The convolution takes C input channels, uses a kernel of size W, and outputs $H_b \times W$ channels. The projection has $D \times C$ parameters and the convolution has $W^2 \times C \times H_b$ parameters; the computation (MACs) per token is the same as the parameter count.

To reduce the convolution parameter count, we introduce three additional hyperparameters:

- Groups: number of convolution groups;
- Share: whether to share weights across convolution groups;
- Rank: rank used in the low-rank convolution.

4 Experiments

All models in our experiments use Multi-Query Attention (MQA)[14], with $K = V$[15]. Each attention layer has $D_{model} \times D_{head} \times (2 \times N_{heads} + 1)$ parameters. Each feed-forward layer uses SwiGLU[16] and has $8 \times D^2_{model}$ parameters.

4.1 Hyperparameter Search

We train each model for one epoch on WikiText-103[17] and record the validation loss. Training uses the Hugging Face Trainer[18], AdamW[19], and the GPT-2 tokenizer[3]. The training hyperparameters are: batch size 4, sequence length 1024, learning rate 3×10^{-4} with cosine learning-rate scheduler, and seed 42.

We train two types of backbone models, a tiny backbone with $N_{layers} = 12$, $D_{model} = 192$, $N_{heads} = 3$, $D_{head} = 64$, and a base backbone with $N_{layers} = 12$, $D_{model} = 768$, $N_{heads} = 12$, $D_{head} = 128$. The model name format is "`backbone RoPE/NoPE` W C Rank H_b Groups Share". For example, "`tiny rope w128d6r128h3g1s`" denotes a tiny backbone using RoPE with $W = 128$, $C = 6$, Rank $= 128$, $H_b = 3$, Groups $= 1$, and Share weights. For simplicity, model names may omit the default configuration: `NoPE`, `r0`(no low-rank convolution), `h1` ($H_b = 1$), `g1`(Groups $= 1$), and `s0` (no weight sharing).

First, we train tiny models with hyperparameters around "`nope w128d6r0h3g1`". Figure 3 shows that all ConvPE variants outperform the RoPE baseline. The largest gains come from increasing the window size and the projection dimension. Using grouped convolution causes the largest performance drop. RoPE integration and other parameter-saving methods show minor differences from "`nope w128d6r0h3g1`".

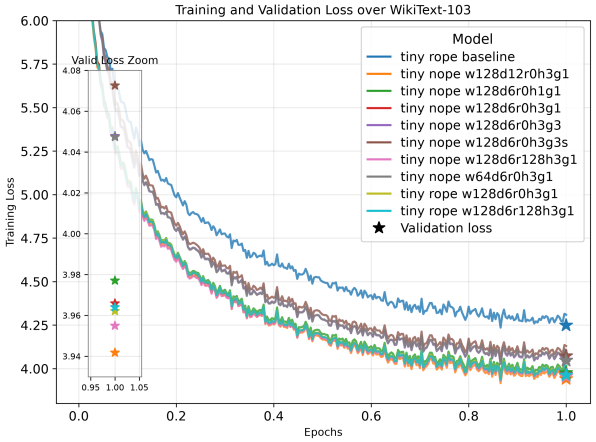


Figure 3: Tiny model training curve on WikiText-103

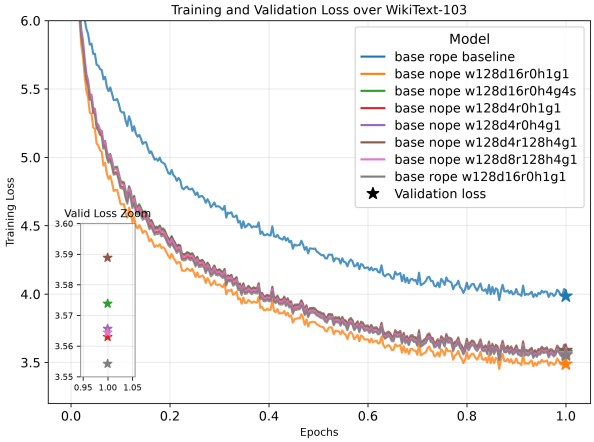


Figure 4: Base model training curve on WikiText-103

We then train base models with hyperparameters around "`nope w128d4r0h4g1`" and "`nope w128d16r0h1g1`". Figure 4 shows that $H_b = 1$ without any parameter-saving technique achieves the best balance between performance and parameter efficiency.

In additional experiments, we also try using X, Q[11] or KV as the convolution input, but performance drops.

4.2 Pretraining

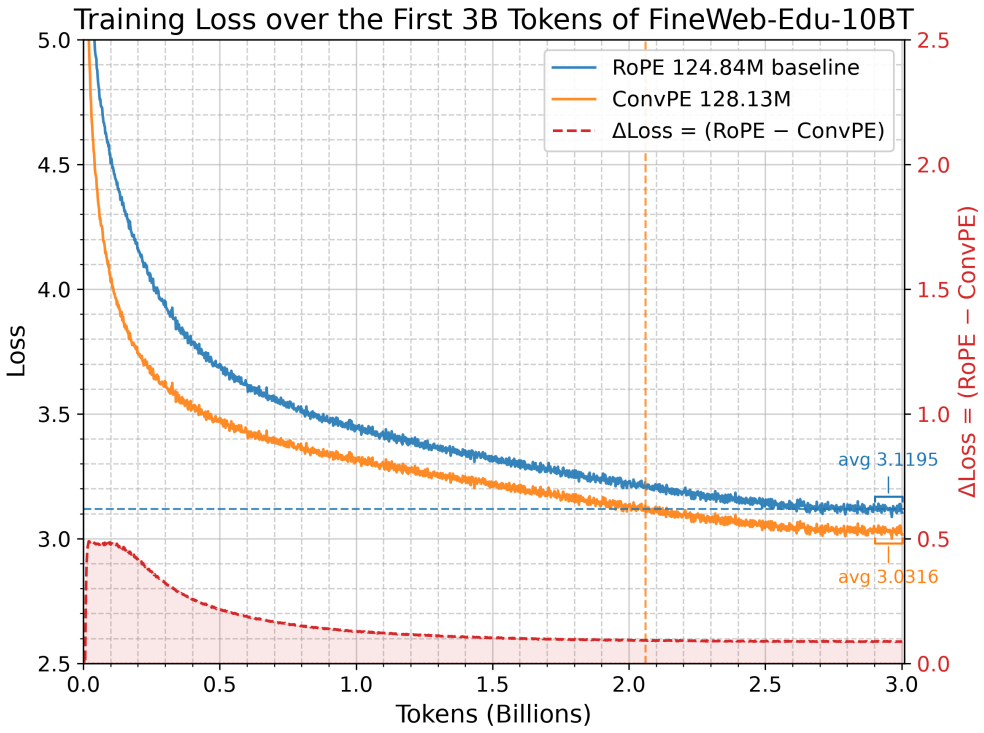


Figure 5: Pretraining curve on the first 3B tokens of FineWeb-Edu-10BT

We choose "`base w128d16`" as the ConvPE variant configuration for performance and throughput considerations. We train the RoPE baseline and the ConvPE variant from scratch on the first 3B tokens of FineWeb-Edu-10BT[20]. We use Muon[21] to optimize the 2D parameters, excluding the embedding layer and the language head, with a learning rate of 3×10^{-3}, while Adam[22] optimizes the remaining parameters at a learning rate of 3×10^{-4}, both using a cosine

learning-rate scheduler. The training hyperparameters are: batch size 4, gradient accumulation steps 4, sequence length 1024, and seed 42.

Figure 5 shows that ConvPE converges faster and achieves a lower final loss compared to the RoPE baseline. We implement high-performance Flash MQA kernels in TileLang and benchmark two configurations: `base RoPE` with 124.84M parameters and `base w128d16` with 128.13M parameters ($+2.64\%$), using a batch size of 4 and a sequence length of 1024. While the forward-pass time increases by 2.71%, slightly exceeding the parameter increase, the final training loss drops from 3.1195 to 3.0316 (-2.82%). Despite the 6.54% per-step overhead, the number of training tokens required to match the RoPE baseline's final loss decreases from 3B to 2.06B, a 31.33% reduction. This translates to a net reduction in total wall-clock training time of approximately 26.87%.

4.3 Attention Analysis

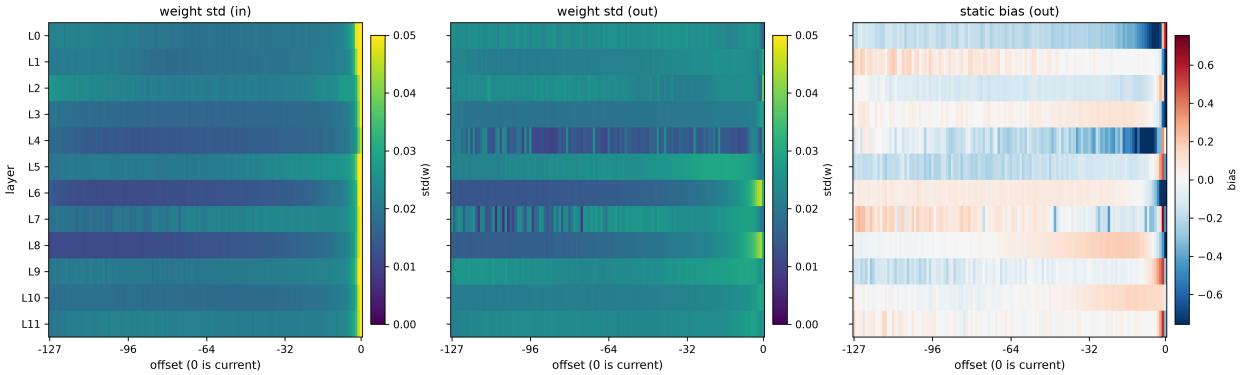


Figure 6: Visualization of ConvPE statistics across layers. Left: weight standard deviation in the input dimension. Middle: weight standard deviation in the output dimension. Right: static output bias. The x-axis represents relative offsets (-127 to 0), where 0 is the current token.

To understand the actual role of the ConvPE module, we analyze the weight standard deviation and the static output bias. Since the average weights are around zero, the standard deviation serves as an indicator of **feature selectivity**: a higher std implies the kernel focuses on specific channels. The numerical analysis reveals three primary properties:

1. Near-End Dominance. Near the current token, both the static bias and the input feature selectivity reach their maximum magnitudes. This indicates that the positional relationship between nearby tokens is crucial. ConvPE reflects this by applying highly specific feature extraction and strong positional priors to the immediate local context.

2. Far-End Relaxation. Towards the far end of the window, the standard deviation becomes flat, and the static bias converges to nearly zero. This shows that ConvPE does not impose strong static biases at this range, delegating the routing decision entirely to the convolution on dynamic input and the QK semantic matching of the tokens themselves.

3. Self-Suppression and Self-Enhancement. The static bias exhibits a clear layer-wise prior at the current position (offset 0). Layer 0 shows "self-enhancement" with a positive bias of $+0.5885$, whereas all deeper layers exhibit "self-suppression" with negative biases. This mechanism shifts attention in deeper layers away from the current token and toward the historical context.

4.4 Evaluations

This section evaluates the pretrained ConvPE and the RoPE baseline on pretraining and context-extension benchmarks without any fine-tuning.

4.4.1 Pretraining Evaluations

Table 1 shows that ConvPE consistently improves language-modeling performance over the RoPE baseline. On WikiText, perplexity drops from 46.56 to 42.05, and on LAMBADA perplexity decreases substantially from 118.10 to 88.49, with accuracy also improving from 0.2507 to 0.2604. ConvPE also outperforms RoPE on most downstream benchmarks. The gains are consistent across most metrics. The main exception is TruthfulQA-MC1, where both models fall below the random-choice baseline, indicating that this benchmark remains difficult for both models. Overall, the pretraining results support ConvPE as a stronger positional encoding than RoPE for the evaluated language tasks.

Dataset	Metric	RoPE	ConvPE	Random
wikitext	perplexity	46.56	**42.05**	-
lambada[23]	perplexity, acc	118.10, 0.2507	**88.49**, **0.2604**	-
hellaswag[24]	acc, acc norm	0.2894, 0.3148	**0.2949**, **0.3272**	0.25
xstorycloze_en[25]	acc	0.5460	**0.5526**	0.5
piqa[26]	acc, acc norm	0.6251, 0.6208	**0.6338**, **0.6235**	0.5
arc_easy[27]	acc, acc norm	0.5114, 0.4487	**0.5421**, **0.4701**	0.25
winogrande[28]	acc	0.5091	**0.5193**	0.5
boolq[29]	acc	0.5676	**0.5893**	0.5
truthfulqa_mc1[30]	acc	0.2399	0.2375	**0.25**

Table 1: Evaluation results of pretraining models

4.4.2 Context Extension Evaluations

Context	RoPE	ConvPE
1K	3.1247	**3.0312**
2K	3.1684	**3.0021**
4K	4.2690	**3.0278**
8K	5.6443	**3.3575**

Table 2: Validation loss on 1M training tokens

We first evaluate language-modeling performance using 1M tokens from the training dataset with increasing context length. Both models are pretrained on 1K-token contexts. Table 2 shows a clear advantage for ConvPE in language modeling as context length increases. RoPE degrades sharply beyond 2K, whereas ConvPE remains comparatively stable up to 4K and degrades much more gracefully at 8K. This indicates that ConvPE provides better context-length extrapolation and preserves model performance under longer contexts.

	1K		2K		4K	
Task	RoPE	ConvPE	RoPE	ConvPE	RoPE	ConvPE
niah_single_1	0.614	**0.890**	0.004	**0.722**	0.000	**0.314**
niah_single_2	0.612	**0.664**	0.000	**0.038**	0.000	0.000
niah_single_3	0.090	**0.180**	0.000	**0.014**	0.000	0.000
niah_multikey_1	0.198	**0.200**	0.118	**0.194**	0.000	0.002
niah_multikey_2	0.002	0.000	–	–	–	–
niah_multikey_3	0.000	0.008	–	–	–	–
niah_multiquery	0.104	**0.186**	0.029	**0.179**	0.000	0.000
niah_multivalue	0.149	**0.187**	0.127	**0.186**	0.000	0.000
ruler_vt	**0.032**	0.008	0.000	**0.052**	0.000	**0.107**
ruler_cwe	0.002	0.002	–	–	–	–
ruler_fwe	0.001	**0.012**	–	–	–	–
ruler_qa_squad	0.148	**0.191**	0.078	**0.091**	0.017	**0.052**

Table 3: RULER evaluations

We then evaluate both models on RULER[31] tasks. Table 3 shows that ConvPE generally improves over RoPE on several long-context retrieval and reasoning tasks, particularly at 1K and 2K contexts. ConvPE even performs well at 4K contexts on some tasks. Some tasks remain near zero for both models, indicating that these tasks are too difficult for 125M-parameter models. Although the language-modeling loss of ConvPE remains stable up to 4K, the downstream tasks show that zero-shot context extension is still far from solved.

4.5 2D Position Embedding

To produce 2D positional embeddings, we use 2D convolutional layers that take a 2D window input and produce a 2D window output. We use ViT-Tiny[5] models with Multi-Head Attention (MHA) and the configuration $N_{\text{layers}} = 12$,

$D_{\text{model}} = 192$, $N_{\text{heads}} = 3$, and $D_{\text{head}} = 64$. We train the models on CIFAR-10 and CIFAR-100[6] for 100 epochs with patch shape 4×4.

model	params	CIFAR-10	CIFAR-100
RoPE baseline	5.346M	0.910	0.627
ConvPE2D `w3d4`	5.359M	0.931	**0.695**
ConvPE2D `w3d8`	5.372M	**0.933**	0.683
ConvPE2D `w5d4`	5.402M	0.929	0.682
ConvNet	4.335M	**0.945**	**0.737**

Table 4: Validation accuracy on CIFAR-10 and CIFAR-100

Table 4 shows that ConvPE transfers effectively to vision transformers and outperforms the RoPE baseline. The influence of hyperparameters requires further study on larger models and datasets.

4.6 Peer Comparison

We thank Qiao Zhijian from Hong Kong University of Science and Technology for evaluating ConvPE with additional baselines. Note that the training setting is not entirely the same as ours; therefore, the results are not directly comparable to ours. The main difference is that the experiments in this section do not tie the language model head weights to the embedding layer weights. Details are reported in the ConvPE GitHub repository (`pe_comparison.md`).

			FineWeb-Edu 1M PPL					RULER Acc		
method	M params	ms/step	1K	2K	4K	8K	Eval Acc	1K	2K	4K
NoPE[12]	163.470	**141.30**	24.34	173.71	1389.25	3618.99	42.78	19.31	0.15	0.00
Composite[11]	163.498	180.02	21.34	38.15	246.16	984.34	44.37	25.97	0.88	0.09
RoPE[4]	163.470	149.03	22.89	24.19	55.01	207.58	43.86	11.96	2.63	0.26
ConvPE	166.764	156.00	20.52	19.95	20.67	29.33	44.66	**26.58**	14.46	3.49
ALiBi[10]	163.470	145.67	21.37	20.77	20.60	20.52	43.90	14.37	3.80	1.61
CABLE[32]	163.691	188.05	20.71	20.01	19.68	19.52	44.72	19.80	7.69	3.22
DAPEv2[33]	163.512	273.71	**19.47**	**18.71**	**18.31**	**18.09**	**44.74**	26.11	**17.09**	**7.45**

Table 5: Comparison with peer methods. Parameters, milliseconds per step, and PPL are lower-is-better metrics, whereas accuracies are higher-is-better. Eval acc denotes the average accuracy in Section 4.4; RULER acc denotes the average accuracy in Table 3, where the normalized accuracy (acc norm) is used when available.

Method	M params	ms/step	CIFAR-100 Acc
NoPE[12]	5.363	66.45	56.03
2D ALiBi[34]	5.363	66.63	61.38
Composite2D[11]	5.596	78.29	63.50
2D RoPE[35]	5.363	78.44	64.17
Learnable 2D abs. PE[5]	5.376	**63.50**	64.36
Fixed 2D sin-cos[5]	5.363	66.65	65.07
CABLE2D[32]	5.377	98.90	65.25
DAPE2Dv2[33]	5.374	92.74	66.01
ConvPE2D	5.377	82.03	**67.98**

Table 6: CIFAR-100 results. ms/step is lower-is-better, and accuracy is higher-is-better.

Table 5 shows that ConvPE is among the first tier of methods in terms of both FineWeb-Edu 1M PPL and evaluation accuracy. It achieves highly competitive PPL across context lengths and remains substantially better than NoPE, Composite, and RoPE at longer contexts, while its evaluation accuracy is very close to the best result. On RULER, ConvPE delivers the best overall performance among the non-quadratic methods, with the strongest accuracy at the training context length and competitive results at medium and long contexts. Although DAPE exhibits stronger length extrapolation, it has quadratic complexity and is much slower per training step, making it impractical for large-scale long-context training. ALiBi and CABLE are efficient, but both impose rigid distance-penalty priors ALiBi uses fixed

linear decay, and CABLE only learns to modulate a fixed distance-penalty skeleton which limits their ability to model ultra-long contexts. In contrast, ConvPE applies NoPE outside the window and therefore does not suffer from an explicit extrapolation limit, offering a more favorable accuracy-efficiency trade-off for large-scale long-context modeling.

Table 6 reports the CIFAR-100 results. ConvPE2D achieves the best accuracy among all compared methods.

5 Conclusions and Future Work

ConvPE delivers substantial performance gains at a minor parameter and computational cost. Although some existing methods report stronger results on individual metrics, these gains are accompanied by compromises in training efficiency, compatibility, or extrapolation behavior. When performance, parameter efficiency, computational cost, and compatibility are considered jointly, ConvPE provides the most favorable overall trade-off in our evaluations and is a strong alternative for positional embeddings.

We adopt $H_b = 1$ without any parameter-saving technique as the default configuration, which achieves the best balance among performance, parameter efficiency, and computational cost. ConvPE is compatible with RoPE, various attention masks, and KV-cache concatenation, and it naturally supports multidimensional extensions.

Future work can explore the following directions:

- **Kernel Optimization.** Further optimize the backward kernel and develop additional kernels for a wider range of hardware.
- **Scaling behavior.** Investigate ConvPE with larger Transformers and datasets, and identify configuration patterns at scale.
- **Long-context modeling.** Evaluate ConvPE in long-context settings, where positional generalization is particularly important.
- **Architectural integration.** Combine ConvPE with Sliding Window Attention (SWA)[36], Compressed Sparse Attention (CSA)[15], and other Transformer variants.
- **Multidimensional modalities.** Extend ConvPE to high-dimensional modalities such as vision, speech, and multimodal data.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[3] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[4] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[6] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[7] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. Tilelang: A composable tiled programming model for AI systems. *CoRR*, abs/2504.17577, 2025. doi:10.48550/ARXIV.2504.17577. URL `https://doi.org/10.48550/arXiv.2504.17577`.

[8] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=wHBfxhZu1u`.

[9] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[10] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

[11] Tyler A. Chang, Yifan Xu, Weijian Xu, and Zhuowen Tu. Convolutions and self-attention: Re-interpreting relative positions in pre-trained language models, 2021. URL `https://arxiv.org/abs/2106.05505`.

[12] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. *arXiv preprint arXiv:2305.19466*, 2023.

[13] Kimi Team. Kimi k3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026. URL `https://arxiv.org/abs/2607.24653`.

[14] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[15] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026. URL `https://arxiv.org/abs/2606.19348`.

[16] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[18] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[19] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[20] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

[21] Keller Jordan. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[22] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL `http://arxiv.org/abs/1412.6980`.

[23] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, 2016. doi:10.18653/v1/P16-1144.

[24] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

[25] Xi Victoria Lin, Todor Mihaylov, Mikel Artetxe, Tianlu Wang, Shuohui Chen, Daniel Simig, Myle Ott, Naman Goyal, Shruti Bhosale, Jingfei Du, Ramakanth Pasunuru, Sam Shleifer, Punit Singh Koura, Vishrav Chaudhary, Brian O'Hare, Jeff Smith, Luke Zettlemoyer, Mike Lewis, Ves Stoyanov, and Sergey Edunov. Few-shot learning with multilingual language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022.

[26] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439, 2020. doi:10.1609/aaai.v34i05.6239.

[27] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[28] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.

[29] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.

[30] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*, 2021.

[31] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024.

[32] Ali Veisi et al. CABLE: Context-aware biases for length extrapolation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. Please verify the exact author list, pages, and URL from the official version.

[33] Chuanyang Zheng, Yihang Gao, Han Shi, Jing Xiong, Jiankai Sun, Jingyao Li, Minbin Huang, Xiaozhe Ren, Michael Ng, Xin Jiang, Zhenguo Li, and Yu Li. DAPE V2: Process attention score as feature map for length extrapolation. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10628–10666, Vienna, Austria, 2025. Association for Computational Linguistics. doi:10.18653/v1/2025.acl-long.522. URL https://aclanthology.org/2025.acl-long.522/.

[34] Anthony Fuller, Koreen Millard, and James R. Green. CROMA: Remote sensing representations with contrastive radar-optical masked autoencoders. In *Advances in Neural Information Processing Systems*, volume 36, pages 5506–5538, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/11822e84689e631615199db3b75cd0e4-Abstract-Conference.html.

[35] Byeongho Heo, Song Park, Dongyoon Han, and Sangdoo Yun. Rotary position embedding for vision transformer. In *European Conference on Computer Vision (ECCV)*, 2024. URL https://arxiv.org/abs/2403.13298.

[36] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.